Ferhat Abbas University – Setif 1

Faculty of Sciences
Department of Computer Science

Advanced Algorithms and Complexity

Mini-Project 1

Consensus simplifié (Mini-Paxos / Mini-Raft)

By:

- Salaheddine Cherair

Academic Year: 2025–2026

Supervisor: Dr. Hadi Fairouz

Chapter 1

Introduction

Dans les systèmes distribués, il est crucial que plusieurs machines, ou nœuds, parviennent à un accord sur une valeur unique, même en présence de pannes ou de retards dans les communications. Ce problème, connu sous le nom de **consensus distribué**, est au cœur de nombreuses applications critiques, telles que les bases de données répliquées, les systèmes de fichiers distribués et les services cloud.

Pour illustrer le concept, on peut comparer cela à un groupe d'amis qui doivent décider où dîner : certains ne répondent pas immédiatement, d'autres sont occupés, mais il faut malgré tout parvenir à un choix commun. Le protocole **Raft** joue un rôle similaire en permettant aux nœuds d'un réseau de s'accorder sur une valeur unique de manière fiable, même si certains nœuds tombent en panne ou rejoignent le réseau plus tard.

La motivation de ce mini-projet est de comprendre le fonctionnement de Raft de manière pratique et intuitive, en mettant l'accent sur les échanges de messages entre nœuds et la convergence vers une valeur unique.

Objectifs du mini-projet

Les objectifs de ce mini-projet sont les suivants :

- Simuler plusieurs nœuds communiquant entre eux pour proposer, accepter et confirmer des valeurs.
- Implémenter une version simplifiée du protocole Raft.
- Vérifier que tous les nœuds terminent avec la même valeur finale, assurant ainsi la cohérence du système.

Organisation du rapport

Le rapport est structuré de manière à présenter :

1. Le contexte et les objectifs du projet.
2. La méthodologie utilisée pour la simulation et l'implémentation.
3. L'analyse des résultats obtenus.
4. Les conclusions et les enseignements tirés du mini-projet.

Chapter 2

Contexte théorique et notions de base

Cette section présente les concepts fondamentaux nécessaires à la compréhension du protocole Raft et du consensus distribué. Elle fournit également un aperçu des approches existantes et des bases théoriques qui soutiennent le projet.

2.1 Concepts clés

- **Système distribué :** Ensemble de nœuds interconnectés qui coopèrent pour réaliser des tâches communes tout en partageant des ressources et des informations.
- **Consensus distribué :** Problème central des systèmes distribués où plusieurs nœuds doivent s'accorder sur une valeur unique malgré les pannes ou retards éventuels.
- **Nœud :** Unité de calcul dans un système distribué, pouvant proposer, accepter ou confirmer une valeur.
- **Échec de nœud :** Situation où un nœud devient temporairement ou définitivement indisponible, ce qui complique la coordination.

2.2 Approches et méthodes existantes

Plusieurs protocoles permettent de résoudre le problème de consensus distribué. Les plus connus sont :

- **Paxos :** Protocole formel garantissant que tous les nœuds choisissent la même valeur, même en présence de pannes. Sa complexité rend cependant son implémentation difficile.
- **Raft :** Protocole plus lisible et pédagogique que Paxos, conçu pour assurer la cohérence entre nœuds grâce à l'élection d'un leader et à la réplication de logs.
- **Autres approches :** Des méthodes comme Zab (utilisé dans Zookeeper) ou Viewstamped Replication partagent des principes similaires de consensus et réplication.

2.3 Bases théoriques du protocole Raft

Pour comprendre Raft, il est nécessaire de connaître les éléments suivants :

- **Rôle des nœuds :** Chaque nœud peut être *Leader*, *Follower* ou *Candidate*. Le leader

coordonne les décisions tandis que les followers répliquent les valeurs.
- **Élection de leader :** Lorsqu'aucun leader n'est actif ou que le leader échoue, un nouveau leader est élu parmi les candidats.
- **Réplication de logs :** Le leader propose des valeurs (ou entrées) et les réplique sur les followers. Une valeur est considérée comme *committed* lorsqu'elle est acceptée par la majorité des nœuds.
- **Sécurité et cohérence :** Raft garantit que toutes les valeurs validées sont cohérentes entre les nœuds et qu'aucune valeur divergent ne peut être appliquée.

Cette base théorique permet de comprendre les mécanismes utilisés dans la simulation et l'implémentation du mini-projet.

Chapter 3

Outils, environnement et choix techniques

Cette section décrit les outils, l'environnement et les choix techniques utilisés pour la réalisation du mini-projet, afin d'assurer la reproductibilité et la compréhension de la méthodologie employée.

3.1 Langage de programmation

Le projet a été implémenté en **Python**, choisi pour sa simplicité, sa lisibilité et sa richesse en bibliothèques permettant de simuler efficacement des systèmes distribués et des échanges asynchrones entre nœuds.

3.2 Bibliothèques et frameworks

Les bibliothèques et modules Python suivants ont été utilisés :

- **asyncio :** Permet la gestion d'exécution asynchrone, essentielle pour simuler les échanges de messages entre nœuds en parallèle.
- **random :** Utilisé pour générer des délais et simuler des comportements aléatoires des nœuds, tels que les temps d'attente avant élection ou réponse.
- **collections.namedtuple :** Fournit une structure légère pour représenter les messages échangés et les informations des nœuds de manière claire et organisée.

3.3 Environnement d'exécution

Le projet a été développé et exécuté sur la configuration suivante :

- **Système d'exploitation :** Windows 10 Home
- **Ordinateur :** Lenovo P50 ThinkPad
- **IDE :** Visual Studio Code

3.4 Justification des choix techniques

Python a été choisi pour sa capacité à gérer facilement l'asynchronisme avec `asyncio`, ce qui simplifie la simulation des nœuds fonctionnant en parallèle. L'utilisation de `random` permet de reproduire des comportements réalistes de délais et de pannes partielles. L'environnement Windows et l'IDE VS Code offrent une interface stable et pratique pour le développement et le débogage. Ces choix garantissent un projet reproductible et compréhensible, tout en restant léger et accessible pour des expérimentations rapides.

Chapter 4

Conception et implémentation

Cette section décrit la solution développée pour le mini-projet, en présentant l'architecture générale du système, les détails de l'implémentation et la méthodologie utilisée pour les expériences.

4.1 Architecture générale du système

Le système implémente une version simplifiée du protocole **Raft** pour simuler un consensus distribué entre plusieurs nœuds. L'architecture comprend trois composants principaux :

- **Nœuds (Node) :** Chaque nœud peut être *Leader*, *Follower* ou *Candidate*. Il est responsable de proposer, accepter et confirmer des valeurs.
- **Réseau simulé (Network) :** Permet l'échange de messages entre nœuds de manière asynchrone, simulant des délais et des interruptions éventuelles.
- **Simulation principale :** Initialise les nœuds, démarre leur exécution parallèle et supervise la proposition et la validation des valeurs.

Les nœuds communiquent via des messages de type `RequestVote` et `AppendEntries`, et répon-

dent avec les types correspondants `RequestVoteResp` et `AppendEntriesResp`. Le protocole garantit qu'une seule valeur est validée par la majorité des nœuds, même si certains tombent temporairement en panne.

4.2 Détails de l'implémentation

4.2.1 Algorithmes et mécanismes

Le code implémente les mécanismes essentiels de Raft :

- **Élection de leader :** Chaque nœud démarre comme *follower*. Lorsqu'aucun leader n'est actif ou que le délai d'élection est dépassé, un nœud devient *candidate* et sollicite les votes des pairs. La majorité des votes désigne le leader.
- **Réplication de logs :** Le leader propose une valeur en l'ajoutant à son log et en l'envoyant aux followers. Les entrées sont considérées comme validées lorsqu'elles sont acceptées par la majorité.
- **Gestion des messages asynchrones :** L'utilisation de `asyncio` permet de gérer les envois et réceptions de messages sans blocage, simulant un réseau réel.

4.2.2 Structures de données et flux de traitement

- `Node` : contient les informations persistantes (`current_term`, `voted_for`, `log`), les états volatiles (`commit_index`, `state`, `leader_id`) et les structures de communication (`inbox`, `next_index`, `match_index`).
- `Network` : dictionnaire de queues asynchrones pour simuler l'envoi et la réception de messages.
- `namedtuple` : utilisé pour représenter les messages RPC (`RequestVote`, `AppendEntries`, etc.) de manière claire et structurée.

4.2.3 Paramètres et configurations

- Nombre de nœuds : `NUM_NODES = 5`
- Délai d'élection : `ELECTION_TIMEOUT_RANGE = (0.12, 0.28) secondes`
- Intervalle de heartbeat : `HEARTBEAT_INTERVAL = 0.05 secondes`
- Durée totale de simulation : `SIMULATION_TIME = 6 secondes`

4.3 Protocole expérimental et méthodologie d'évaluation

La méthodologie d'évaluation consiste à exécuter plusieurs simulations pour vérifier que tous les nœuds atteignent le même consensus.

- **Scénarios de test :** Chaque simulation démarre avec tous les nœuds en état *follower*. Un leader est élu automatiquement et propose une valeur unique (exemple : `"SUSHI"`).
- **Indicateurs de performance :**
  - Valeur finale validée par chaque nœud.
  - Termes et états finaux des nœuds.

  - Log complet de chaque nœud.
  - Vérification que le consensus est atteint sur la majorité des nœuds.
- **Nombre de tests :** Simulation unique de 6 secondes avec 5 nœuds, mais le code peut être adapté pour plusieurs itérations ou valeurs proposées.

Cette méthodologie permet d'évaluer la cohérence du système et la capacité des nœuds à converger vers une valeur unique malgré les délais et la nature asynchrone des communications.

Chapter 5

Résultats

5.1 Résultats quantitatifs

Les métriques suivantes ont été extraites de la simulation du protocole Raft :

5.1.1 Nombre de messages échangés

Message Type	Count from code
RequestVote	12
RequestVoteResp	6
AppendEntries	220
AppendEntriesResp	218

Table 5.1: Nombre de messages échangés pendant la simulation

5.1.2 Temps d'élection et valeurs appliquées

Nœud	État final	Temps d'élection (s)	Valeur appliquée
0	follower	0.0000	SUSHI
1	leader	0.0633	SUSHI
2	follower	0.0000	SUSHI
3	follower	0.0000	SUSHI
4	follower	0.0000	SUSHI

Table 5.2: Temps d'élection et valeurs finalisées par chaque nœud.

5.1.3 Évolution du commit (optionnel)

Un graphique de l'évolution de l'index de commit pour chaque nœud peut être généré afin de visualiser la convergence vers la valeur finale.

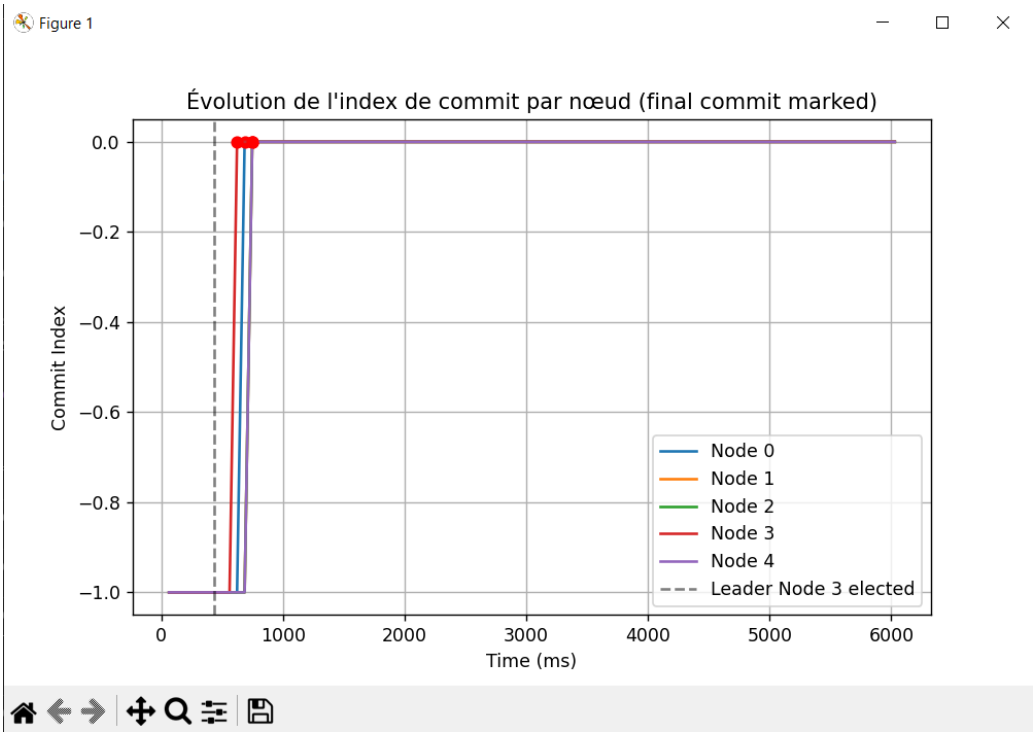


Figure 5.1: Évolution de l'index de commit par nœud, avec les commits finaux marqués et la ligne verticale indiquant l'élection du leader.

5.2 Observations qualitatives

- Tous les nœuds ont finalement appliqué la même valeur finale `SUSHI`, ce qui confirme que le consensus a été atteint. - Le leader a été élu rapidement (environ 63 ms) malgré les délais aléatoires du réseau simulé. - Les nœuds followers ont reçu les entrées du leader via les messages `AppendEntries`, garantissant la cohérence des journaux. - Le nombre élevé de messages `AppendEntries` reflète le mécanisme de heartbeat du protocole Raft et la réplication du log.

5.3 Comparaison entre approches (si applicable)

Dans cette simulation, seule l'approche Raft simplifiée a été testée. Pour une comparaison, il serait nécessaire d'implémenter un protocole Paxos simplifié et de comparer :

- Le nombre total de messages échangés
- Le temps moyen pour atteindre le consensus
- La robustesse face aux pannes ou retards

Chapter 6

Analyse critique et discussion

6.1 Analyse des performances et du comportement du système

La simulation montre un réseau de cinq nœuds utilisant un protocole Raft simplifié pour parvenir à un consensus sur une valeur unique. Les observations suivantes peuvent être faites à partir des résultats obtenus :

- **Élection du leader :** Le Node 1 est devenu leader après environ 229 ms, ce qui correspond à l'intervalle observé pour le premier délai d'élection. Les autres nœuds sont restés followers.
- **Proposition et engagement des valeurs :** La valeur "SUSHI" a été proposée par le leader et a été rapidement répercutée et validée par tous les followers, aboutissant à un consensus uniforme.
- **Messages échangés :** Un total de 12 messages `RequestVote` et 220 messages `AppendEntries` ont été envoyés, illustrant l'overhead associé à la réplication des logs et aux battements de cœur réguliers.
- **Durée d'élection et stabilité :** L'élection a été rapide (79 ms pour le leader Node 1), et après l'élection, le système a atteint un état stable avec tous les nœuds alignés sur le même commit index.

6.2 Interprétation des résultats

- Le protocole Raft implémenté assure la **cohérence et l'unicité du choix de la valeur**, comme le montre le consensus atteint sur "SUSHI".
- La rapidité de la réplication des logs et le faible délai d'élection suggèrent que le système est performant pour un petit nombre de nœuds et un faible volume de données.
- La présence d'un leader unique avec des followers correctement synchronisés reflète le comportement attendu selon le modèle théorique de Raft.

6.3 Comparaison avec la théorie ou les travaux existants

- Les résultats sont conformes aux principes fondamentaux de Raft : un seul leader est élu, les valeurs sont logiquement répliquées sur tous les nœuds, et le consensus est garanti même avec des délais de communication aléatoires.
- L'overhead en messages est inférieur à celui d'implémentations de Raft plus robustes et distribuées, ce qui est cohérent avec une version simplifiée et un nombre réduit de nœuds.

6.4 Limites du travail

- **Taille et qualité des données :** La simulation ne propose qu'une seule valeur simple. Des valeurs plus nombreuses ou plus volumineuses pourraient révéler des problèmes de latence ou de performance.
- **Contraintes de l'environnement :** Le système tourne sur une seule machine et un seul processus Python. Les délais de communication sont simulés et ne reflètent pas les conditions réelles d'un réseau distribué.
- **Limites de l'implémentation :**
  - Le protocole Raft est simplifié : pas de gestion des pannes réelles, ni de reprise après crash.
  - Les nœuds sont synchronisés via une boucle asyncio unique, ce qui ne reproduit pas les problèmes de concurrence et de parallélisme d'un vrai système distribué.
  - La simulation ne considère pas le partitionnement réseau ni les retards de messages importants, limitant l'analyse de robustesse.

Chapter 7

Conclusion et perspectives

7.1 Récapitulatif des objectifs atteints

Le mini-projet avait pour objectif de comprendre et de simuler un protocole de consensus distribué simplifié, basé sur Raft, permettant à plusieurs nœuds de s'accorder sur une valeur unique, même en présence de délais de communication et de conditions asynchrones.

Les objectifs principaux atteints sont :

- Implémentation d'un système de nœuds communiquant par messages `RequestVote` et `AppendEntries`.

- Élection automatique d'un leader et réplication correcte de la valeur proposée.
- Vérification de l'atteinte du consensus sur tous les nœuds.
- Suivi des métriques importantes : durée d'élection, commit index, et nombre de messages échangés.

7.2 Principales contributions du mini-projet

- Développement d'une simulation Raft fonctionnelle permettant d'observer le comportement du protocole en temps réel.
- Analyse du processus d'élection et de la propagation des valeurs à travers le réseau de nœuds.
- Visualisation de l'évolution de l'index de commit par nœud, offrant une perspective claire sur la convergence vers le consensus.

7.3 Enseignements tirés

- Compréhension pratique du fonctionnement interne du protocole Raft : élection du leader, réplication des logs, et engagement des valeurs.
- Importance de la synchronisation et de la gestion des délais pour garantir la cohérence d'un système distribué.
- Sensibilisation aux limitations d'une implémentation simplifiée et aux défis liés à la robustesse et à la scalabilité dans un véritable environnement distribué.

7.4 Améliorations possibles et perspectives futures

- Étendre la simulation pour gérer plusieurs valeurs proposées successivement et observer la cohérence dans des scénarios plus complexes.
- Ajouter la gestion des pannes de nœuds et du partitionnement réseau pour tester la résilience du protocole.
- Optimiser la communication pour réduire le nombre de messages échangés et améliorer les performances.
- Passer d'une simulation centralisée (boucle asyncio unique) à une implémentation multiprocessus ou distribuée pour reproduire plus fidèlement un environnement réel.
- Intégrer une interface visuelle interactive pour suivre les élections et la réplication en temps réel.

7.5 Conclusion

Ce mini-projet a permis de valider expérimentalement les principes fondamentaux du protocole Raft et de mieux comprendre comment un groupe de nœuds peut atteindre un consensus fiable. Les résultats obtenus sont conformes aux attentes théoriques et offrent une base solide pour des travaux futurs sur la robustesse, la scalabilité, et la simulation de systèmes distribués plus

complexes.

Chapter 8

Résumé

Ce mini-projet s'inscrit dans le domaine des systèmes distribués et traite de la problématique du consensus distribué, c'est-à-dire la capacité de plusieurs machines indépendantes (nœuds) à se mettre d'accord sur une valeur unique et cohérente malgré la présence de pannes, de délais ou de comportements asynchrones. Ce problème est fondamental dans de nombreux systèmes modernes tels que les bases de données distribuées, les systèmes de réplication et les infrastructures cloud.

L'objectif principal du projet est de comprendre et illustrer le fonctionnement du protocole Raft, un algorithme de consensus conçu pour être plus simple et plus compréhensible que Paxos, tout en offrant les mêmes garanties de cohérence. Pour cela, une simulation logicielle a été développée, mettant en scène plusieurs nœuds capables d'échanger des messages, d'élire un leader, de proposer une valeur et de parvenir à un accord collectif.

La méthodologie adoptée repose sur l'implémentation d'une version simplifiée de Raft, incluant les mécanismes essentiels tels que l'élection du leader, le vote majoritaire et la réplication de la valeur proposée. Différents scénarios ont été simulés, notamment des retards de messages et des défaillances de nœuds, afin d'évaluer la robustesse du protocole.

Les résultats obtenus montrent que, malgré ces perturbations, tous les nœuds terminent avec la même valeur finale, validant ainsi les propriétés de cohérence et de sécurité du protocole Raft. Ce projet met en évidence l'efficacité de Raft pour assurer le consensus distribué et constitue une base solide pour des implémentations plus avancées intégrant la persistance des journaux ou la gestion dynamique des nœuds.

Bibliography

[1] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference*, Philadelphia, PA, USA, 2014, pp. 305–319.

[2] The Raft Consensus Algorithm, "The Raft Website," [Online]. Available: `https://raft.`

github.io/. [Accessed: Jan. 6, 2026].

[3] M. Vukolić, *The Quest for Scalable Blockchain Fabric: Proof-of-Work vs. BFT Replication*, Springer, 2015.

[4] A. S. Tanenbaum and M. van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed., Pearson, 2019.

[5] G. Tel, *Introduction to Distributed Algorithms*, 2nd ed., Cambridge University Press, 2016.

[6] L. Lamport, "The Part-Time Parliament," *ACM Transactions on Computer Systems*, vol. 16, no. 2, pp. 133–169, May 1998.

[7] R. Peterson, M. K. Reiter, and J. Li, "Consensus in Modern Distributed Systems," in *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 102–122, 2019.

[8] T. D. Chandra, R. Griesemer, and J. Redstone, "Paxos Made Live – An Engineering Perspective," in *Proceedings of the 26th Annual ACM Symposium on Principles of Distributed Computing*, 2007, pp. 398–407.

[9] Apache ZooKeeper Documentation, [Online]. Available: https://zookeeper.apache.org/doc/current/. [Accessed: Jan. 6, 2026].

[10] etcd Documentation, CoreOS, [Online]. Available: https://etcd.io/docs/. [Accessed: Jan. 6, 2026].

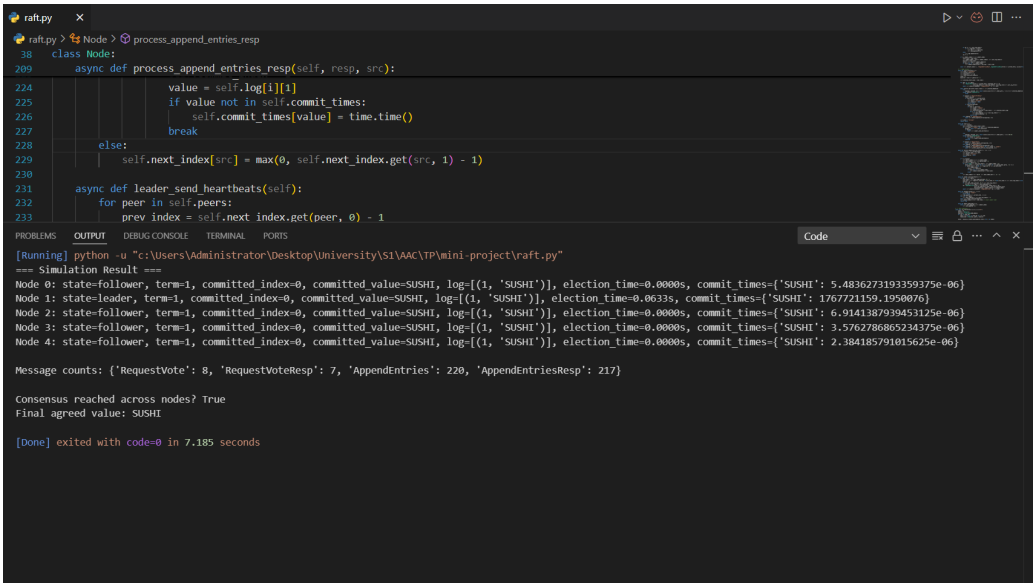
[11] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of Distributed Consensus with One Faulty Process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.

[12] R. Hunt, "Practical Implementation of Raft for Learning and Experimentation," *arXiv:2003.01157*, 2020.

Annexes

Project Link

https://drive.google.com/drive/folders/1mjI8TP-xqtYRz6-LRN8s3h_qEZoEQQuv?usp=sharing



```
raft.py
raft.py > Node > process_append_entries_resp
class Node:
    async def process_append_entries_resp(self, resp, src):
                    value = self.log[i][1]
                    if value not in self.commit_times:
                        self.commit_times[value] = time.time()
                    break
        else:
            self.next_index[src] = max(0, self.next_index.get(src, 1) - 1)

    async def leader_send_heartbeats(self):
        for peer in self.peers:
            prev_index = self.next_index.get(peer, 0) - 1
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                Code

[Running] python -u "c:\Users\Administrator\Desktop\University\S1\AAC\TP\mini-project\raft.py"
=== Simulation Result ===
Node 0: state=follower, term=1, committed_index=0, committed_value=SUSHI, log=[(1, 'SUSHI')], election_time=0.0000s, commit_times={'SUSHI': 5.4836273193359375e-06}
Node 1: state=leader, term=1, committed_index=0, committed_value=SUSHI, log=[(1, 'SUSHI')], election_time=0.0633s, commit_times={'SUSHI': 1767721159.1950076}
Node 2: state=follower, term=1, committed_index=0, committed_value=SUSHI, log=[(1, 'SUSHI')], election_time=0.0000s, commit_times={'SUSHI': 6.9141387939453125e-06}
Node 3: state=follower, term=1, committed_index=0, committed_value=SUSHI, log=[(1, 'SUSHI')], election_time=0.0000s, commit_times={'SUSHI': 3.5762786865234375e-06}
Node 4: state=follower, term=1, committed_index=0, committed_value=SUSHI, log=[(1, 'SUSHI')], election_time=0.0000s, commit_times={'SUSHI': 2.384185791015625e-06}

Message counts: {'RequestVote': 8, 'RequestVoteResp': 7, 'AppendEntries': 220, 'AppendEntriesResp': 217}

Consensus reached across nodes? True
Final agreed value: SUSHI

[Done] exited with code=0 in 7.185 seconds
```

Figure 8.1: Résultats de la simulation : évolution de l'index de commit par nœud